Beyond Generic AI: Build Custom Agents in Jupyter AI with QuickAgent

by *Sanjiv Das*

Jupyter AI v3.0 brings a paradigm-shifting concept to our favorite data science notebooks: Personas. Here, we present QuickAgent, one persona to rule all agents.

Instead of interacting with a single, generic chat assistant, Jupyter AI allows you to summon specialized AI collaborators into your sidebar using @ mentions. You can talk to a standard LLM, deploy a specialized software harness like Claude Code, or summon a custom agent tailored to your exact data domain.

If you have spent any time watching autonomous AI agents work, you know the feeling. You give one a task — "analyze this dataset and write me a paper" — and then you just sit back and watch. It reads files, runs code, searches the web, fixes its own errors, and hands you a finished product. No scaffolding. No babysitting. Just results. The first time I saw this work end-to-end, I thought: *this changes everything about how researchers work.*

The trouble is, building an agent from scratch is still, for most people, a software engineering exercise. You wrestle with APIs, tool registrations, prompt templates, and environment variables before you ever get to the interesting part. That friction is what makes implementation difficult for the scientists, educators, and analysts who need it most.

This is the problem that `jupyter-ai-quickagent` is designed to solve. It is a lightweight Jupyter AI submodule. QuickAgent allows data scientists to build their own agents dynamically through a chat-based interaction.

What Is QuickAgent?

QuickAgent is a Jupyter AI *persona* — a named assistant that lives inside the JupyterLab chat panel. Unlike a generic chatbot, a persona can be extended with custom behavior, tools, and domain knowledge. QuickAgent uses that extension point to let you build, save, and run your own autonomous agents through a simple guided conversation.

QuickAgent was built by looking at the Jupyternaut persona, and extending it with agent-creation capabilities.

You type `@QuickAgent create` into the Jupyter AI chat, answer five questions, and your agent is ready to run.

Why is this useful?

Jupyter AI already gives you Jupyternaut, a capable assistant that can explain code, generate notebooks, and work with your files. What does an agent layer add?

Most IDE-native AI assistants force you into a one-size-fits-all model. If you are working on a highly specialized task—such as analyzing climate data or engineering complex financial pipelines—a

generic LLM lacks the specialized tools, system prompts, and context required to be genuinely helpful.

While Jupyter AI 3.0 allows for multi-agent workflows, manually coding a new agent persona from scratch every time you switch projects introduces unwanted friction.

The answer is *autonomy* and *specialization*, both provided by QuickAgent.

- Autonomy: A standard chat assistant waits for you to direct every step. An agent is given a goal — "analyze all the files in this folder and produce a LaTeX research paper," and then it plans, acts, observes the results, and iterates until the goal is achieved. It uses tools: a Python REPL, file I/O, web search, shell commands. It recovers from failures and keeps going until the work is done.
- Specialization: A general-purpose agent is fine for work that is one-off and not intended to be repeated. But if you want an agent that always generates mathematically rigorous exam questions, or one that writes research papers following your specific methodology, you need to inject domain knowledge. QuickAgent does this through *skills* — a folder of Markdown files containing instructions, workflows, and domain context that get woven into the agent's system prompt automatically.

The combination of autonomy plus specialization, configured without code, is what makes this useful. Instead of configuring endless JSON files or writing intricate Python wrappers to define an agent's skills, tools, and behavior, QuickAgent focuses on a chat-based interactive approach to agent creation. You tell Jupyter what you need your persona to do, and QuickAgent helps manifest it as an active, mentionable selection right inside the QuickAgent persona.

How It Works

QuickAgent is structured to sit seamlessly alongside your Jupyter AI installation. QuickAgent is built on top of LangChain Deep Agents, a framework for building tool-using agents. It plugs into Jupyter AI through the persona manager, which means it inherits Jupyter AI's model and credential configuration automatically. You set up your API key once in **Settings > Jupyternaut Settings**, and every agent you build uses it.

Authentication is handled. What remains is defining *what your agent does*.

When you run `@QuickAgent create`, a five-step wizard asks you:

1. **What do you want to call this agent?**
2. **What is its purpose?** (A plain-English description — "analyze research datasets and write papers")
3. **Which tools should it have access to?** (File I/O, Python REPL, shell execution, web fetch, and more)
4. **Which search tools?** (DuckDuckGo is free and built in; Tavily, Wikipedia, arXiv, and PubMed are optional)
5. **Does it have a skills directory?** (Point it to a folder of `.md` skill files, and their contents are injected into the system prompt)

The result is saved as a JSON file in `~/.jupyter/jupyter-ai/quickagents/`. This ensures the agent you created is persistent and reusable. You can edit it by hand if you like.

To run it, type: `@QuickAgent use <agent-name>` followed by any prompt. The agent takes it from there.

QuickAgent makes Data Science easy

Instead of data scientists writing all the code, their role is evolving into that of a principal investigator orchestrating an elite team of specialized AI agents. You might have one agent that specializes in fetching remote geospatial data formats, another that excels at cleaning data frames, and a third designed to optimize machine learning hyperparameters.

The traditional boundaries between writing code and instructing an AI are blurring. QuickAgent highlights a critical trend: agentic hierarchies. By lowering the barrier to entry for creating these personas, repositories like `jupyter-ai-quickagent` are making specialized AI accessible to everyday developers—no complex backend engineering required.

Two Examples

The MathQ Agent

A colleague wrote an essay on what makes a good mathematics question — the kind that tests deep conceptual understanding rather than mechanical computation. We converted this essay into a `SKILL.md` file and pointed a QuickAgent at it.

The result is an agent that generates exam questions aligned to those criteria. It does not just produce any question — it produces questions it can defend as *good* questions, explaining why each one tests reasoning rather than recall. Whether it always gets this right is a matter of judgment. But the framework — injecting expert human criteria into an agent's decision-making via a skills file — is a useful pattern. You can find the MathQ agent and its associated skills file here.

10:17 AM

@QuickAgent use MathQ

QuickAgent 10:17 AM

Switched to agent: MathQ

Generate and solve math questions that test deep conceptual understanding rather than mechanical computation or memorized formulas. These questions should reward creativity, multi-perspective thinking, and the ability to connect seemingly unrelated ideas.

Send me a message and I'll process it with this agent. Say `create` to make a new agent or `list` to see all agents.

10:18 AM

Question on the RSA algorithm

QuickAgent 10:18 AM

Using skills: `skills/SKILL.md`

I'll generate a conceptually rich RSA question following the skill guidelines.---

Question

RSA encryption uses a public key (n, e) and private key (n, d), where $n = pq$ for large primes p, q, and $ed \equiv 1 \pmod{(p-1)(q-1)}$.

Suppose Alice encrypts a message m as $c = m^e \bmod n$ and sends it to Bob. Charlie, an eavesdropper, somehow learns **both** the public key (n, e) and the private key d. Charlie wants to factor n to mount future attacks on other RSA keys.

Claim: Knowing d (and the public key (n, e)) is *sufficient* to factor n efficiently with high probability — even though d reveals no direct information about p or q individually.

(a) Explain why knowing $ed - 1$ gives you a "handle" on the structure of $(\mathbb{Z}/n\mathbb{Z})^*$, the multiplicative group of integers mod n. What is $ed - 1$ divisible by, and why is that useful?

(b) Here is a probabilistic algorithm sketch: Write $ed - 1 = 2^s \cdot t$ with t odd. Pick a random a with $\gcd(a, n) = 1$, and consider the sequence

$$a^t,\ a^{2t},\ a^{4t},\ \ldots,\ a^{2^s t} \equiv 1 \pmod{n}.$$

Somewhere in this sequence, there is a "first 1." Explain why the step *just before* that first 1 is likely to be a **non-trivial square root of 1 mod n** — and why that immediately gives you a factorization of n.

(c) A student argues: *"This algorithm can't always work, because $a^t \equiv 1 \pmod{n}$ is possible right from the start, giving no useful information."* Is this a fatal flaw? Estimate roughly how often this happens and what should be done.

Answer

(a) Since $ed \equiv 1 \pmod{(p-1)(q-1)}$, we have:

$$ed - 1 = k(p-1)(q-1) \quad \text{for some positive integer } k.$$

By Fermat's Little Theorem (or Lagrange's theorem), for any a coprime to n:

$$a^{(p-1)(q-1)} \equiv 1 \pmod{p} \quad \text{and} \quad a^{(p-1)(q-1)} \equiv 1 \pmod{q}.$$

Therefore $a^{ed-1} \equiv 1 \pmod{n}$. So $ed - 1$ is a **known multiple of the exponent of the group** $(\mathbb{Z}/n\mathbb{Z})^*$. This is powerful: it tells us that every element of the group, raised to this known power, equals 1 — giving us a structured "window" into the group's order without knowing p or q directly.

(b) The key insight is about **square roots of** 1 **modulo** $n = pq$.

By CRT, $x^2 \equiv 1 \pmod{n}$ has **four** solutions:

$$x \equiv \pm 1 \pmod{p}, \quad x \equiv \pm 1 \pmod{q}.$$

Two of these are $x \equiv \pm 1 \pmod{n}$ (trivial). The other two are *non-trivial*: e.g., $x \equiv 1 \pmod{p}$ but $x \equiv -1 \pmod{q}$.

Now consider the squaring sequence. We know $a^{2^r t} \equiv 1 \pmod{n}$. Let $a^{2^{r-1}t}$ be the last term before the sequence first hits 1. Then:

$$\left(a^{2^{r-1}t}\right)^2 \equiv 1 \pmod{n}, \quad \text{but} \quad a^{2^{r-1}t} \not\equiv 1 \pmod{n}.$$

So $a^{2^{r-1}t}$ is a **non-trivial square root of** 1 **mod** n. Call it x. Then:

$$x \not\equiv \pm 1 \pmod{n}, \quad x^2 \equiv 1 \pmod{n}.$$

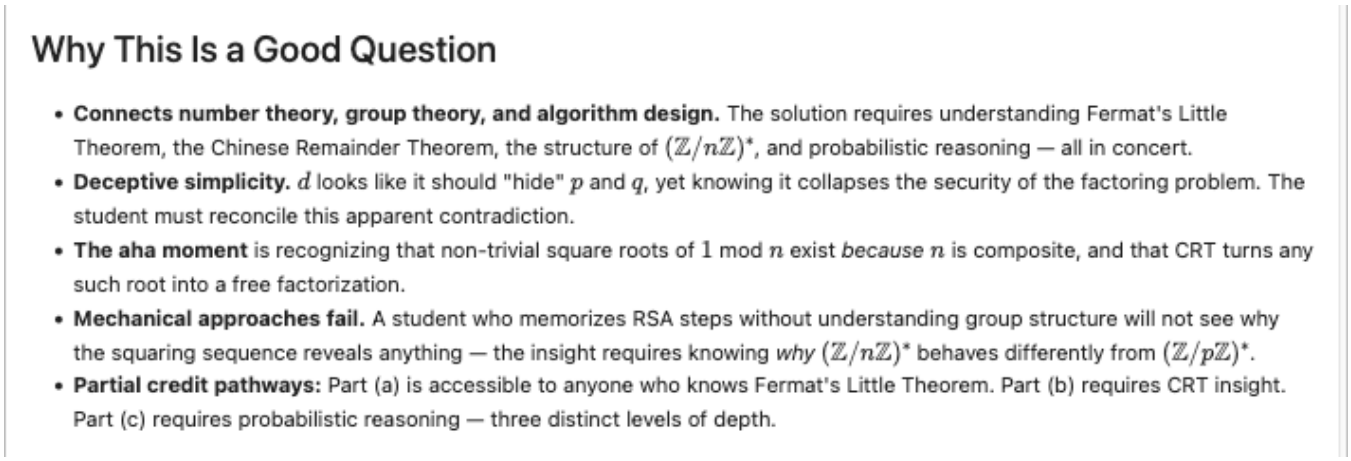
This means $(x-1)(x+1) \equiv 0 \pmod{n}$, but neither factor is $0 \pmod{n}$. So $\gcd(x-1, n)$ must be exactly p or q — a nontrivial factor. **One GCD computation cracks** n**.**

(c) This is **not** a fatal flaw — it's a manageable nuisance.

The sequence starting with a^t reaches 1 at the *very first step* only if $a^t \equiv 1 \pmod{p}$ **and** $a^t \equiv 1 \pmod{q}$ simultaneously. By CRT and the structure of $\mathbb{Z}/p\mathbb{Z}^*$, the probability that a random a satisfies $a^t \equiv 1 \pmod{p}$ is $\gcd(t, p-1)/(p-1)$, which is at most $1/2$ (since $p-1$ is even and t is odd, so $\gcd(t, p-1)$ divides the odd part of $p-1$, which is at most $(p-1)/2$). A symmetric argument applies mod q.

More carefully: at each squaring step, the chance of landing in the "trivial" case is roughly $1/2$ or less per prime. So the algorithm fails for a given a with probability at most $1/2$, and **repeating with a fresh random** a drives the failure probability down exponentially. In practice, a handful of trials suffices.

The deeper point: **the algorithm is probabilistic but efficient** — polynomial time with overwhelming success probability. This is a hallmark of modern cryptographic algorithms where randomness is a feature, not a bug.



Why This Is a Good Question

- **Connects number theory, group theory, and algorithm design.** The solution requires understanding Fermat's Little Theorem, the Chinese Remainder Theorem, the structure of $(\mathbb{Z}/n\mathbb{Z})^*$, and probabilistic reasoning — all in concert.
- **Deceptive simplicity.** d looks like it should "hide" p and q, yet knowing it collapses the security of the factoring problem. The student must reconcile this apparent contradiction.
- **The aha moment** is recognizing that non-trivial square roots of 1 mod n exist *because* n is composite, and that CRT turns any such root into a free factorization.
- **Mechanical approaches fail.** A student who memorizes RSA steps without understanding group structure will not see why the squaring sequence reveals anything — the insight requires knowing *why* $(\mathbb{Z}/n\mathbb{Z})^*$ behaves differently from $(\mathbb{Z}/p\mathbb{Z})^*$.
- **Partial credit pathways:** Part (a) is accessible to anyone who knows Fermat's Little Theorem. Part (b) requires CRT insight. Part (c) requires probabilistic reasoning — three distinct levels of depth.

The Researcher Agent

You point the Researcher agent at a folder containing a CSV dataset and any other files (paper, dscriptions of the data, etc.). The agent reads the data, formulates research questions, writes and executes Python code to produce statistical analyses and visualizations, generates a reproducible Jupyter notebook, and produces a complete LaTeX paper.

The output is first-pass research work, not just a summary, but tables, figures, statistical tests, and a structured paper you could work from as a first draft. The skills directory for this agent contains a set of `.md` files defining what good empirical research looks like, what a rigorous paper structure entails, and how to handle common data quality issues. The agent consults all of it and prepares the outputs, leveraging Jupyter notebooks to provide fully reproducible results.

We are at a moment where AI can genuinely accelerate scientific work, not by replacing judgment, but by handling the execution layer. The researcher does not even need to define the question, or design the methodology, evaluates the output, but must take responsibility for the conclusions. But the hours of boilerplate coding, formatting, and mechanical analysis? Those can be delegated to the agent and verification and checking becomes an important human exercise. Of course, the human can influence all parts of this process.

The bottleneck has always been the gap between "I have a dataset" and "I have a result." QuickAgent narrows that gap. A well-designed agent, given good skills and clear direction, can turn a dataset into a draft paper in the time it used to take to just set up the analysis environment. The Researcher example in this repository does exactly that, with documented output you can inspect yourself. You can also see additional examples of generated research in the `examples` folder.

The deeper point is this: the most valuable thing about configuring agents with *tools* and *skills* is that it encodes expertise. An experienced researcher's intuitions about what constitutes a good analysis, what a rigorous paper looks like, what pitfalls to avoid — all of that can be written down in Markdown and given to an agent. The agent then applies it consistently, at scale, across every task. You are not just automating work.

It is not only about research. Any agent can be spun up quickly. Here we only highlighted two agents, one that generates good math questions, and the other that does autonomous research.

An invitation to Jupyter AI

It is worth being explicit about the layering here. Jupyter AI is the foundation, an extensible AI integration for JupyterLab that handles model configuration, the chat UI, magic commands, and the persona manager. QuickAgent is an add-on to that foundation, implementing one specific persona focused on autonomous agent creation and execution.

If you are not already using Jupyter AI, the QuickAgent installer sets everything up for you. If you are, QuickAgent slots in alongside Jupyternaut and any other personas you have installed. The two complement each other well — Jupyternaut for conversational code assistance, QuickAgent for autonomous multi-step tasks.

The full Jupyter AI documentation is at jupyter-ai.readthedocs.io/en/v3/ and is worth reading to understand the broader ecosystem.

Get Started

The repository is at github.com/srdas/jupyter-ai-quickagent. Clone it, follow the installation instructions in the README, and you can get up and running.

Option 1: Full JupyterLab Install (Recommended)

Install uv first — it is the fastest Python package manager available and makes the rest of this trivial.

Then download the installer script and run it:

```
chmod a+x install_nondev.sh
./install_nondev.sh
```

That single script installs Jupyter AI, all its dependencies, and QuickAgent, then launches JupyterLab. For subsequent sessions:

```
cd jupyter-ai-quickagents
source .venv/bin/activate
jupyter lab
```

Once JupyterLab is open, configure your model in **Settings > Jupyternaut Settings** (remember it is based on Jupyternaut). Any provider supported by LiteLLM works — OpenAI, Anthropic, Google, Azure, AWS Bedrock, and dozens more. Then open the chat panel, and start building quick agents.

Option 2: CLI-Only Install

If you want to run agents from the terminal without JupyterLab, the CLI install is just as straightforward:

```
git clone https://github.com/srdas/jupyter-ai-quickagent.git
cd jupyter-ai-quickagent
uv venv --python 3.12
```

```
source .venv/bin/activate
uv pip install -r CLI-requirements.txt
```

Connect it to a model:

```
export QUICKAGENT_MODEL='us.anthropic.claude-opus-4-6-v1'
```

Create your first agent:

```
python -m jupyter_ai_quickagent create
```

Run it:

```
python -m jupyter_ai_quickagent run MyAgent "Summarize the key findings in this folder"
```

Your First Agent in Three Commands (JupyterLab)

Once you are inside the JupyterLab chat panel:

```
@QuickAgent create
```

Follow the five prompts. Then activate your new agent:

```
@QuickAgent use MyAgent
```

And put it to work:

```
Analyze the data in ~/Downloads/survey_results and write a summary report
```

That is it. The agent plans its approach, reads the files, runs the analysis, and writes the report — all without further input from you.

This is just "One persona to rule all agents."